

Prompt 4: Generate the After Trigger for Table R

Specification-to-code compilation contract for an LLM.

Consider the formal framework for incremental maintenance of RDB2RDF views defined in the first attached file: **IVM_abril_21__Current_version – versão resumida : 2027_EDBT em anexo**

- Consider **LinkedBrainz RDF View** that is an RDF-based semantic view of the MusicBrainz relational database.
- The relational schema of the MusicBrainz is in file: **MusicBrainz Schema**. The data is stored in PostgreSQL. The file **MB_TRs** is the **single source of truth** for the definition of the schema of MusicBrainz.
- The ontology for **LinkedBrainz RDF View** is in file: **MusicbrainzOntoUltimate.owl**

Storing the Materialized RDF view

- GraphDB stores the materialized RDF view as a dataset of quads:
(s, p, o, g), where $g = \text{URI}(\Psi)$

- Each transformation rule Ψ corresponds to a named graph:
GRAPH <URI(Ψ)>
- The RDF dataset is accessible via SPARQL endpoint:
- The pre-update RDF state W_0 MUST be obtained from GraphDB via SPARQL

=====

Transformation Rules:

- The **LinkedBrainz RDF View** is defined by a set of object-preserving Transformation Rules (TRs) in file: **MB_TRs**. The file **MB_TRs** is the **single source of truth** for the definition of the LinkedBrainz RDF View.

- Use only the Transformation Rules (TRs) explicitly defined in **MB_TRs**.
- Do not use, adapt, extrapolate, or complete rules from examples appearing in papers, documentation, conversations, or previous outputs.
- Do not assume the existence of additional rules that are not present in **MB_TRs**.
- If a rule is not explicitly defined in **MB_TRs**, treat it as non-existent.
- All maintenance computations must be derived exclusively from the TRs in **MB_TRs**.

The examples shown in the paper are provided only for explanatory purposes and do not represent the complete LinkedBrainz RDF View specification. Therefore, they must never be used as the basis for maintenance computations or code generation

URI Construction Function used by the TRs :

Table 1 in **MB_TRs** defines the URI construction functions used by the Transformation Rules. These functions specify how RDF resource URIs are generated from tuples in the relational database.

For example, the first row defines the function:

```
URI_artist(a, x) := hasURI("http://musicbrainz.org/artist/", [a.gid,
"#_"], x)
```

This means that the URI x is constructed by concatenating:

1. the URI prefix "http://musicbrainz.org/artist/",
2. the value of the attribute $a.gid$,
3. the suffix "#_".

Whenever a Transformation Rule contains an atom of the form:

$URI_X(t, u)$ the LLM must:

1. Locate the definition of URI_X in Table 1 of **MB_TRs**.
2. Apply the corresponding $hasURI(...)$ function exactly as specified.
3. Generate URIs using the defined prefix, attributes, ordering, separators, and suffixes.
4. Never infer URI patterns from examples, resource names, or previous outputs.
5. Treat Table 1 as the single authoritative source for URI generation.

If a URI function is not defined in Table 1, the LLM must report that the definition is missing rather than inventing a URI pattern.

Transformation Rule URIs

Each Transformation Rule Ψ is identified by a URI.

The URI of a Transformation Rule with identifier ψ_id is constructed by concatenating:

1. the fixed URI prefix: "http://musicbrainz.org/graph/mapping/"
2. the transformation rule identifier ψ_id .

Formally:

$URI(\Psi) =$

"http://musicbrainz.org/graph/mapping/" || $rule_id(\Psi)$

The named graph URI associated with Ψ is the same URI:

$URI_G(\Psi) = URI(\Psi)$

Example:

For the transformation rule: psi_artist_01 the rule URI is:

http://musicbrainz.org/graph/mapping/psi_artist_01

and the named graph containing the RDF contribution generated by this rule is also:

GRAPH <http://musicbrainz.org/graph/mapping/psi_artist_01>

=====

TASK

Consider the table $R = \text{Track}$.

Generate the PostgreSQL statement-level AFTER trigger function **Compute_Changeset_R** responsible for computing RDF incremental maintenance information for updates on a relation R .

The generated function MUST implement the incremental maintenance architecture and semantics defined in:

- Algorithm 1
- Algorithm 2
- Section 6
- Formal semantics of transformation rules
- Relevance definitions

from the document **IVM_abril_21__Current_version**.

Asynchronous architecture for incremental maintenance of a materialized RDB2RDF view.

The **incremental maintenance of a materialized RDB2RDF view** will be executed in a PostgreSQL trigger-based architecture using transition tables and an asynchronous RDF maintenance queue as described in [**Infrastructure for the Maintenance Queue**].

The asynchronous architecture is composed of:

1. PostgreSQL statement-level AFTER triggers;
2. A maintenance queue (rdf_maintenance_queue);
3. An external worker that processes events and interacts with GraphDB.

INPUT ASSUMPTIONS

Assume :

- $u = (D, I)$ is an update over relation R
- $D = \text{deleted_R}$
- $I = \text{inserted_R}$

where:

- deleted_R is the transition table containing deleted/pre-update tuples
- inserted_R is the transition table containing inserted/post-update tuples

Relevance of Transformation Rules:

Before generating the function $\text{Compute_Changeset_R}$, the **LLM MUST** first identify the set of transformation rules relevant to updates on relation R , according to the formal relevance definitions provided in **IVM_abril_21__Current_version**.

– Ψ is pivot-relevant iff $\text{pivot}(\Psi) = R$

– Ψ is relation-relevant to updates on relation R iff R occurs in the body of Ψ in an occurrence other than the pivot occurrence. Equivalently, R participates in the relational path of Ψ as an intermediate or target relation, rather than as the initial relation corresponding to $\text{pivot}(\Psi)$.

Formally, a transformation rule Ψ is relevant to relation R iff: Ψ is pivot-relevant or Ψ is relation-relevant.

A transformation rule may be:

- pivot-relevant only
- relation-relevant only
- both pivot-relevant and relation-relevant.

=====

Function $\text{Compute_Changeset_R}$

The function $\text{Compute_Changeset_R}$ consists of two main STEPS as follows (see Algorithm 2 in **IVM_abril_21__Current_version**)

For each transformation rule: $\Psi \in \text{Relev}(R)$ the function MUST compute:

Step 1: Incremental Maintenance Computation

$\Delta + \text{pivot}[\Psi](u) := \emptyset$

$\Delta+rel[\Psi](u) := \emptyset$

1. If Ψ is pivot-relevant

- compute $\Delta+pivot[\Psi](u)$

2. if Ψ is relation-relevant,

- the affected tuples for deletion: $A-[\Psi](u)$
- the affected tuples for insertion: $A+[\Psi](u)$
- the post-update RDF contributions: $S2[\Psi](u)$ used for computing the deletion changeset: $\Delta-[\Psi](u)$
- the relation-relevant insertion changeset: $\Delta+rel[\Psi](u)$

3. Compute the insertion changeset $\Delta+[\Psi](u) = \Delta+pivot[\Psi](u) \cup \Delta+rel[\Psi](u)$

The two relevance conditions are independent and MUST be evaluated separately. If both conditions hold, both computations MUST be executed.

The computations MUST follow the formal semantics and Definitions in Section 5.2 of **IVM_abril_21__Current_version**.

- $S2[\Psi](u)$ is the post-update RDF contribution used by the asynchronous worker to compute:

$$\Delta-rel[\Psi](u) = S1[\Psi](u) \setminus S2[\Psi](u) \text{ where } S1[\Psi](u) \text{ is obtained from } W0.$$

STEP 2: Insertion of events in the Maintenance Queue

The computed maintenance information MUST insert in infrastructure with 2 tables as described in [**Infrastructure for the Maintenance Queue**] .

- `rdf_maintenance_queue` (1 row for event)
- `rdf_rule_contribution`: (1 row for event by rule Ψ)

The tables have the following schema and semantics.

Queue Table: `rdf_maintenance_queue`

Each row represents one statement-level relational update event over a source relation.

The queue preserves the commit order of maintenance events and serves as the communication interface between:

- PostgreSQL AFTER triggers

- the asynchronous RDF maintenance worker

Schema of `rdf_maintenance_queue`

```
CREATE TABLE rdf_maintenance_queue (
  event_id      BIGSERIAL PRIMARY KEY,
  relation_name TEXT NOT NULL,
  operation_type TEXT NOT NULL,
  deleted_tuples JSONB NOT NULL,
  inserted_tuples JSONB NOT NULL,
  status        TEXT NOT NULL DEFAULT 'pending',
  created_at    TIMESTAMP NOT NULL DEFAULT now(),
  processed_at  TIMESTAMP,
  error_message TEXT
);
```

Schema of `rdf_rule_contribution`

```
CREATE TABLE rdf_rule_contribution (
  event_id      BIGINT NOT NULL REFERENCES rdf_maintenance_queue(event_id),
  rule_id       TEXT NOT NULL,
  rule_graph_uri TEXT NOT NULL,
  relevance_type TEXT NOT NULL,
  pivot_relation TEXT NOT NULL,
  path          JSONB NOT NULL,
  rule_contribution JSONB NOT NULL,
  PRIMARY KEY (event_id, rule_id)
);
```

Each row in `rdf_rule_contribution` represents the maintenance information computed for a single transformation rule: $\Psi \in \text{Relev}(\mathcal{R})$, including:

- the affected tuples for deletion: $A-[\Psi](u)$
- the affected tuples for insertion: $A+[\Psi](u)$
- the post-update RDF contributions: $S2[\Psi](u)$
- the insertion changeset: $\Delta+[\Psi](u)$

The rule contributions of each rule should be stored in JSON format, and should follow the templates presented in [**Infrastructure for the Maintenance Queue**] .

The JSON format MUST follow what is defined in the template. Do not change the structure.

The rule contribution MUST preserve the semantics and provenance of the corresponding transformation rule Ψ , including its associated named graph:

=====

OUTPUT FORMAT

A) Analytical Table (MANDATORY)

Provide one row per $\Psi \in \text{Relev}(\text{TRACK})$:

| Ψ | pivot(Ψ) | path(Ψ) | Type (pivot/relation) | Affected tuples | Justification |

Requirements:

- Use the formal definition of relevance
 - Clearly explain how TRACK appears in path(Ψ)
-

B) PostgreSQL Function Compute_Changeset (MANDATORY)

The function MUST follow the algorithm 2 and definitions in **IVM_abril_21__Current_version.**

The function MUST:

- Use transition tables:
deleted_R
inserted_R
 - Use SQL to compute:
 - the affected tuples for deletion $A-[\Psi](u)$,
 - the affected tuples for insertion $A+[\Psi](u)$,
 - the post-update contributions $S2[\Psi](u)$ to $\Delta-(u)$, and
 - the insertion changeset $\Delta+(u)$.
 - Store the computed information in the maintenance queue (rdf_maintenance_queue).
 - Represent RDF quads explicitly:
(s, p, o, g)
 - Include:
 - Clear step-by-step comments
 - Separation of pivot and relation contributions
 - No redundant operations
 - No speculative logic
-

C) PostgreSQL Trigger (MANDATORY)

The AFTER trigger to be executable in a PostgreSQL trigger-based environment using the proposed architecture.

Define all auxiliary table to store the output

FINAL REQUIREMENTS

The function **Compute Changeset_R** MUST address also the 2 phases and return the two tables `rdf_maintenance_queue` and `rdf_rule_contribution`.

The generated solution MUST:

- be fully executable
- be semantically aligned with IVM_FULL
- follow the formal incremental maintenance theory
- avoid informal approximations
- avoid missing cases
- avoid unnecessary abstractions
- explicitly align every SQL fragment with the corresponding formal definition.